AI TRANSIT PIPELINE

Sécuriser l'intégration du code IA en entreprise

- Récupération sécurisée depuis GitHub
- Scan multicouche adaptatif par type de fichier
- Archive ZIP approuvée + rapport Excel de traçabilité

Sommaire

01 Contexte & objectifs

02 Architecture du pipeline

03 Phase 1 — Récupération sécurisée

04 Phase 2 — Scan multicouche

05 Détail des checks par type

06 Outils utilisés

07 Résultats & livrables

08 Règles de sécurité absolues

01 Contexte & Objectifs

Pourquoi ce pipeline ?

- Le code IA généré peut contenir des backdoors, secrets ou patterns dangereux
- Les développeurs importent du code sans revue de sécurité systématique
- Le réseau d'entreprise doit rester isolé (air-gap partiel)
- Aucune traçabilité sans processus structuré

Objectifs clés

Récupérer

Clone sécurisé depth 1

Scanner

Multicouche adaptatif

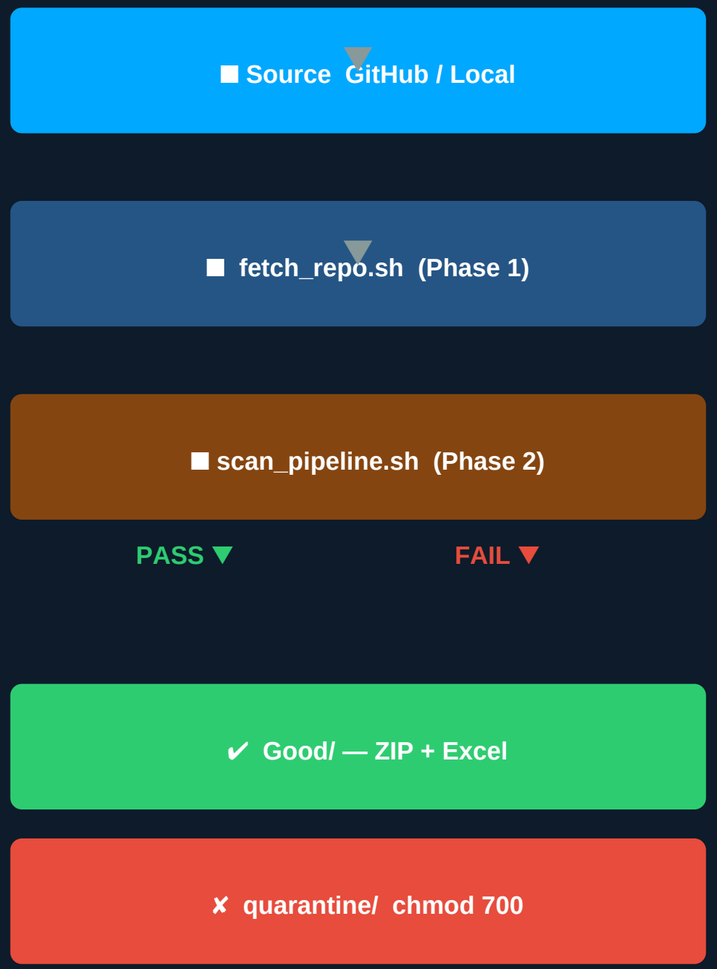
Décider

PASS→ZIP / FAIL→Quarantaine

Tracer

Excel + JSON + HTML

02 Architecture du pipeline



Phase 1 — fetch_repo.sh

- Whitelist hôtes (github.com uniquement)
- Vérif. taille via API GitHub (< 500 MB)
- git clone --depth 1 --no-tags
- Suppression .git/ (pas de metadata)
- Manifest SHA-256 de tous les fichiers
- Triage rapide patterns suspects

Phase 2 — scan_pipeline.sh

Couche GLOBALE :

Gitleaks · detect-secrets · ClamAV · YARA

Couche PAR TYPE :

.py → Bandit + eval/exec

.js/.ts → Semgrep + child_process

.sh → ShellCheck + curl|bash

.yml → actions non-pinnées

.tf → checkov + secrets hardcodés

Dockerfile → hadolint + :latest

.so/.exe → FAIL auto + strings

03 Phase 1 — Récupération sécurisée

1 Whitelist hôtes

Seul github.com autorisé.

Toute autre URL rejetée immédiatement.

2 Vérification taille

API GitHub interrogée avant clone.

Limite : 500 MB.

3 Clone minimal

git clone --depth 1 --no-tags

--single-branch

4 Suppression .git/

Métadonnées Git supprimées

(hooks, remotes, submodules).

5 Manifest SHA-256

Hash de chaque fichier →

.manifest_sha256.txt (audit).

6 Triage rapide

Grep : eval(exec(curl|bash

rm -rf → alerte immédiate.

04 Phase 2 — Couche globale

S'applique à TOUT le répertoire, quel que soit le type de fichier

■ Gitleaks

- > 150 types de credentials
- Tokens GitHub/AWS/GCP/Azure
- Clés RSA, PEM, SSH, JWT
- Analyse entropique + regex

FAIL si positif

■ detect-secrets

- Entropie de Shannon
- Secrets inconnus des règles
- Base64/hex à haute densité
- Complémentaire Gitleaks

FAIL si positif

■ ClamAV

- Millions de signatures
- Trojans, ransomwares, backdoors
- Scan récursif complet
- Base freshclam mise à jour

FAIL si positif

■ YARA

- Standard IOC industrie
- Règles .yar personnalisées
- Patterns backdoors IA
- Regex + logique booléenne

FAIL si positif

04 Phase 2 — Dispatch par type de fichier

Extension(s)	Outil SAST	Checks manuels	Verdict
.py	Bandit	eval() exec() import os/subprocess	FAIL
.js .ts .jsx	Semgrep	eval() child_process require(…)	FAIL
.c .cpp .h	cppcheck	gets() strcpy() system() popen()	FAIL
.sh .bash .ps1	ShellCheck	curl|bash wget|sh eval $var	FAIL
.yml .yaml	—	Actions non-pinnées secrets inline	FAIL
.tf .tfvars	checkov	Secrets en dur IAM trop larges	FAIL
Dockerfile	hadolint	:latest ADD http:// RUN curl|bash	FAIL
.so .exe .elf	strings	FAIL automatique + IOC strings	FAIL ■
.zip .tar.gz	—	FAIL auto — re-scan extraction requis	FAIL ■
.sql	—	xp_cmdshell DROP TABLE ; --	FAIL
.json .md .txt	—	password: secret: token: api_key=	FAIL

05 Détail des vérifications par type

Python .py

Exécution dynamique — risque élevé de backdoors via eval/exec

Bandit SAST

FAIL

AST Python : subprocess shell=True, pickle, MD5, SQL, assert…

eval() / exec()

FAIL

Exécution arbitraire à l'exécution — backdoor triviale

import os/subprocess

WARN

Accès shell système — nécessite revue manuelle

JavaScript .js .ts

Écosystème exposé aux supply-chain attacks

Semgrep p/javascript

FAIL

XSS, prototype pollution, innerHTML, SSTI…

eval()

FAIL

Exécution dynamique — vecteur injection classique

child_process

FAIL

Exécution commandes shell depuis Node.js

05 Détail des vérifications par type

C / C++ .c .cpp

Risques liés à la gestion manuelle de la mémoire

cppcheck

FAIL

Buffer overflows, null ptr, use-after-free, mémoire non init.

gets() strcpy()

FAIL

Lecture/copie sans limite de taille → buffer overflow

system() popen()

FAIL

Exécution commandes shell → injection possible

Shell .sh .bash

Exécution directe — obfuscation de commandes facile

ShellCheck

FAIL

Variables non quotées, globbing, redirections dangereuses…

curl | bash

FAIL

Exécution code distant sans vérification d'intégrité

eval $variable

FAIL

Injection triviale si variable contrôlée par attaquant

05 Détail des vérifications par type

YAML / CI .yml

Pipelines CI/CD — une mauvaise config ouvre des backdoors

Actions non-pinnées

FAIL

uses: action@main → vulnérable au tag-hijacking

Secrets inline

FAIL

password: valeur (pas ${{ secrets.XXX }})

Terraform .tf

IaC — peut provisionner des ressources cloud entières

checkov (CIS)

FAIL

Buckets S3, IAM trop larges, BDD sans chiffrement at-rest

Secrets en dur

FAIL

password = "valeur" dans .tf/.tfvars

05 Checks par type — Dockerfile · Binaires · Archives · SQL

Dockerfile

- **hadolint** — FAIL
  CIS Docker : :latest, ADD http://, apt sans --no-install-recommends
- **RUN curl|bash** — FAIL
  Téléchargement + exécution sans contrôle d'intégrité

Binaires .so .exe .elf

- **FAIL automatique** — FAIL
  Aucun binaire dans un repo IA — implant/RAT potentiel
- **strings + IOC** — FAIL
  /bin/sh /etc/passwd exec reverse shell URLs détectés

Archives .zip .tar.gz

- **FAIL automatique** — FAIL
  Contenu non scannable sans extraction préalable
- **Zip-slip** — FAIL
  Extraction vers chemins arbitraires (../../etc/cron.d/)

SQL .sql

- **xp_cmdshell** — FAIL
  Exécution commandes shell depuis SQL Server → IOC critique
- **DROP TABLE / --** — FAIL
  Instructions destructrices + pattern injection SQL

06 Outils utilisés — Synthèse

Tous optionnels en dégradé — WARN si absent, FAIL uniquement sur finding actif

Outil	Rôle	Installation	Scope
Gitleaks	secrets/tokens	GitHub binary	Global
detect-secrets	entropie élevée	pip install	Global
ClamAV	malware signatures	apt install clamav	Global
YARA	IOC personnalisés	apt install yara	Global
Bandit	SAST Python	pip install bandit	.py
Semgrep	SAST multi-lang	pip install semgrep	.js .ts
cppcheck	analyse statique C/C++	apt install cppcheck	.c .cpp
ShellCheck	lint shell	apt install shellcheck	.sh .bash
hadolint	lint Dockerfile	GitHub binary	Dockerfile
checkov	sécurité IaC	pip install checkov	.tf .hcl
jq	parsing JSON API GitHub	apt install jq	Utilitaire

07 Résultats — Archive & Rapport Excel

■ Archive ZIP → Good/

```
Good/
■■■ repo_20260615_143022.zip
    ■■■ src/
    ■   ■■■ [code source complet]
    ■■■ README.md
    ■■■ scan_report_20260615.xlsx
```

- ✔ Produite uniquement si verdict PASS
- ✔ Nom horodaté — unicité garantie
- ✔ .manifest_sha256.txt exclu
- ✔ Rapport Excel inclus directement
- ✔ Prête au transfert réseau interne
- ✔ Optionnel : chiffrement GPG

■ Rapport Excel (.xlsx)

Onglet 0 — Résumé

Dépôt / Source	URL GitHub ou chemin local
Date du scan	Horodatage ISO 8601
Hash SHA-256	Empreinte globale de tous les fichiers
Verdict	PASS (vert) / FAIL (rouge)
Compteurs	PASS · WARN · FAIL

Onglet 1 — Fichiers (une ligne/fichier)

#	Fichier	Type	Statut	Message
1	src/main.py	.py	FAIL	bandit:HIGH
2	config.sh	.sh	WARN	shellcheck absent
3	README.md	.md	PASS	—

Trié FAIL→WARN→PASS | Filtre auto | Ligne 1 figée

08 Règles de sécurité absolues

Ces règles NE PEUVENT PAS être modifiées — elles constituent le modèle de menace

Isolation réseau

La machine de transit n'a JAMAIS accès direct au réseau d'entreprise.

Flux unidirectionnel

Seul approved/ est lisible depuis l'interne — pas fetch/, logs/, quarantine/.

Quarantaine root-only

chmod 700 — aucun utilisateur applicatif ne peut lire les fichiers refusés.

Pas de déplacement manuel

Jamais quarantine/ → approved/ sans re-scan complet du pipeline.

Binaires = FAIL absolu

Aucun .so/.exe/.elf dans un repo IA — zéro exception.

Archives = re-scan obligatoire

.zip/.tar.gz → extraction + re-scan dans un environnement isolé dédié.

Exploitation et assurance qualité

Options d'exécution

`--quiet / --verbose`
Verbosité des journaux

`--min-severity`
Seuil de blocage : low | medium | high | critical

`--since COMMIT`
Mode différentiel : uniquement les fichiers modifiés

`--report-only`
Ne bloque jamais, ne met rien en quarantaine

`--no-zip / --no-excel`
Rapports seuls, sans archive

Le pipeline se vérifie lui-même

Suite de tests
666 assertions, sans aucun outil de scan requis

Corpus de règles
chaque finding doit viser le bon fichier

Garde-fous
le code sûr ne doit jamais être signalé

Intégration continue
lint · test · pins · docker

Validation par mutation
le défaut est réintroduit, le test doit échouer

Contrôles par dépôt

.transitignore — motifs gitignore, fichiers exclus de toutes les couches

.transit-allow.json — rétrograde un FAIL connu en WARN, avec justification

Environnement isolé (--offline)

Semgrep, trivy, ClamAV — règles et bases pré-positionnées

pip-audit, safety, npm audit — aucun mode hors ligne : l'absence est consignée

Bloc coverage : indique, couche par couche, si elle a réellement été exécutée

Une suite qu'on n'a jamais vue échouer n'apporte aucune preuve.

Deux tests de cette suite passaient sur du code notoirement défectueux : les tests eux-mêmes étaient faux.

En résumé

Source

GitHub (whitelist)

ou chemin local

Fetch

Clone minimal

.git supprimé · SHA-256

Scan

Couche globale

+ 11 types de fichiers

PASS

ZIP dans Good/

avec Excel inclus

FAIL

Quarantaine

chmod 700 + rapport

Pipeline bash · Mode dégradé · Outils optionnels · Rapports horodatés

github.com/Gaillotte/Claude — branche claude/vigilant-carson-f8twy0